

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2019 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2019	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Liste simplu înlănțuite	2
2. Rețele de calculatoare: Clasificare și topologii	2
II. Rezolvare Titularizare Varianta 3 (17 iulie 2019)	2
SUBIECTUL I	2
1. Metoda Backtracking	2
2. Normalizarea bazelor de date	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Perechi rezervă cuvinte	3
2. Algoritm eficient: Secvență de sumă maximă cu lungime minim 2 (Kadane $O(N)$ timp, $O(1)$ memorie)	5

I. Rezolvare Titularizare Model 2019

SUBIECTUL I (30 de puncte)

1. Structura de date: Liste simplu înlănțuite

- **Definire:** O listă simplu înlănțuită constă în noduri alocate dinamic în memorie, unde fiecare nod conține valoarea informației sale și adresa următorului nod în listă (urm), ultimul nod indicând adresa nulă (nullptr / nil).
- **Operații:** Inserare la început/sfârșit, ștergere nod, parcurgere secvențială.
- **Problemă (Gestiune stivă):**

```
#include <iostream>
using namespace std;
struct Nod {
    int info;
    Nod* urm;
};
void push(Nod* &varf, int x) {
    Nod* nou = new Nod{x, varf};
    varf = nou;
}
int main() {
    Nod* varf = nullptr;
    push(varf, 5); push(varf, 10);
    for(Nod* c = varf; c != nullptr; c = c->urm) cout << c->info << " ";
    return 0;
}
```

2. Rețele de calculatoare: Clasificare și topologii

- **Clasificare:** LAN (Local), MAN (Metropolitan), WAN (Wide Area).
- **Topologii:** Topologia stea (cu un nod central/switch), topologia inel (token ring), topologia magistrală (bus).

II. Rezolvare Titularizare Varianta 3 (17 iulie 2019)

[Subiect PDF](file:

SUBIECTUL I

1. Metoda Backtracking

- **Principiu:** Backtracking este o metodă de programare aplicabilă problemelor unde soluția se poate reprezenta sub forma unui vector $X = (x_1, x_2, \dots, x_n)$ unde fiecare componentă x_i aparține unei mulțimi finite. Metoda construiește soluția pas cu pas, testând la fiecare pas condițiile de validare intermediare. Dacă pasul curent nu poate fi validat, se renunță la el și se revine la pasul anterior (backtrack).
- **Implementare iterativă (Permutări) și Implementare recursivă (Combinări):** Detalierea algoritmilor și generarea datelor.

2. Normalizarea bazelor de date

- **Concepte:** Entitate (obiect din lumea reală), Atribut (proprietate), Identificator unic (Cheie primară).
- **Forme Normale:**
 - **1FN:** Toate atributele sunt atomice (nu există grupuri repetitive).
 - **2FN:** Este în 1FN și toate atributele non-cheie depind complet de cheia primară (nu există dependențe parțiale față de o cheie compusă).
 - **3FN:** Este în 2FN și nu există dependențe tranzitive (atribute non-cheie ce depind de alte atribute non-cheie).

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Perechi rezervă cuvinte

Soluție C++:

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
using namespace std;

int rezerva(const string &s1, const string &s2) {
    if (s1.length() != s2.length() || s1 == s2) return 0;
    int L = s1.length();
    for (int i = 1; i < L; ++i) {
        string temp = s1.substr(i) + s1.substr(0, i);
        if (temp == s2) return 1;
    }
    return 0;
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string cuvant;
    vector<string> cuvinte;
    while (ss >> cuvant) {
        cuvinte.push_back(cuvant);
    }

    int gasit = 0;
    int n = cuvinte.size();
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (rezerva(cuvinte[i], cuvinte[j])) {
                cout << "(" << cuvinte[i] << "#" << cuvinte[j] << ")\n";
                gasit = 1;
            }
        }
    }
}
```

```

    if (!gasit) cout << "nu exista\n";
    return 0;
}

```

Soluție Pascal:

```

program PerechiRezerva;
var
    text_input, cuvant: string;
    cuvinte: array[1..100] of string;
    nr_cuvinte, i, j, gasit: integer;

function rezerva(s1, s2: string): integer;
var
    L, idx: integer;
    temp: string;
begin
    if (length(s1) <> length(s2)) or (s1 = s2) then
    begin
        rezerva := 0;
        exit;
    end;
    L := length(s1);
    for idx := 1 to L - 1 do
    begin
        temp := copy(s1, idx + 1, L - idx) + copy(s1, 1, idx);
        if temp = s2 then
        begin
            rezerva := 1;
            exit;
        end;
    end;
    rezerva := 0;
end;

begin
    readln(text_input);
    text_input := text_input + ' ';
    nr_cuvinte := 0;
    cuvant := '';
    for i := 1 to length(text_input) do
    begin
        if text_input[i] <> ' ' then
            cuvant := cuvant + text_input[i]
        else
        begin
            if length(cuvant) > 0 then
            begin
                nr_cuvinte := nr_cuvinte + 1;
                cuvinte[nr_cuvinte] := cuvant;
                cuvant := '';
            end;
        end;
    end;

    gasit := 0;
    for i := 1 to nr_cuvinte do

```

```

begin
  for j := i + 1 to nr_cuvinte do
    begin
      if rezerva(cuvinte[i], cuvinte[j]) = 1 then
        begin
          writeln('(', cuvinte[i], '#', cuvinte[j], ')');
          gasit := 1;
        end;
      end;
    end;

  if gasit = 0 then
    writeln('nu exista');
end.

```

2. Algoritm eficient: Secvență de sumă maximă cu lungime minim 2 (Kadane O(N) timp, O(1) memorie)

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2019.in");
    long long prev_val, curr_val;
    if (!(fin >> prev_val)) return 0;

    long long k_prev = prev_val;
    long long max_sum = -2e18;

    while (fin >> curr_val) {
        long long m_curr = max(prev_val + curr_val, k_prev + curr_val);
        max_sum = max(max_sum, m_curr);
        k_prev = max(curr_val, k_prev + curr_val);
        prev_val = curr_val;
    }
    fin.close();

    cout << max_sum << endl;
    return 0;
}

```

Soluție Pascal:

```

program MaxSecventa;
var
    fin: text;
    prev_val, curr_val, k_prev, m_curr, max_sum: int64;
begin
    assign(fin, 'titu2019.in');
    reset(fin);
    if eof(fin) then exit;
    read(fin, prev_val);

```

```
k_prev := prev_val;
max_sum := -9999999999999999;

while not eof(fin) do
begin
    read(fin, curr_val);

    if prev_val + curr_val > k_prev + curr_val then
        m_curr := prev_val + curr_val
    else
        m_curr := k_prev + curr_val;

    if m_curr > max_sum then
        max_sum := m_curr;

    if curr_val > k_prev + curr_val then
        k_prev := curr_val
    else
        k_prev := k_prev + curr_val;

    prev_val := curr_val;
end;
close(fin);

writeln(max_sum);
end.
```